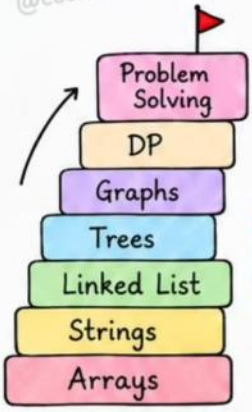
Learn • Practice • Solve • Succeed



DSA

Pattern - Wise

Complete Notes

15 Important Patterns with Examples & Code

★ Best Notes for Interview & Competitive Exams ★

What you will learn ?

- ★ 15 Important DSA Patterns
- ★ Pattern Identification
- ★ Java Code Templates
- ★ Step by Step Explanation
- ★ Examples & Dry Run
- ★ Time & Space Complexity
- ★ Important Interview Questions
- ★ Best Practices & Tips

List of Patterns (15 Pages)

1. Arrays & Hashing
2. Two Pointers
3. Sliding Window
4. Prefix Sum
5. Binary Search
6. Fast & Slow Pointers
7. Linked List Manipulation
8. Stack & Monotonic Stack
9. Binary Tree (DFS & BFS)
10. Heap & Top K
11. Intervals & Greedy
12. Backtracking
13. Graph (BFS & DFS)
14. Dynamic Programming
15. Bit Manipulation

Follow @code.withnarayan
Comment "DSA"
Get Link in Your DM

1. Arrays & Hashing Pattern

★ Use HashMap / HashSet to store and find elements efficiently ★

What is this Pattern ?

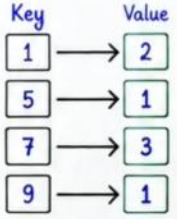
- ★ Used when we need to find, count or check elements in O(1) time.
- ★ Array + HashMap / HashSet helps in solving many problems efficiently.
- ★ Common in interviews and competitive programming.

When to use ?

- ☑ Find if an element exists
- ☑ Count frequency of elements
- ☑ Find duplicates
- ☑ Find pair with given sum
- ☑ Group elements / remove duplicates
- ☑ Problems involving fast lookups

@code.withnarayan

Hashing (Key → Value)



@code.withnarayan

Example Problem : Two Sum

Given an array nums and a target sum, return indices of two numbers such that they add up to target.

@code.withnarayan

Example:

Input : nums = [2, 7, 11, 15], target = 9
Output : [0, 1]
Explanation : nums[0] + nums[1] = 2 + 7 = 9

Approach (Using HashMap)

1. Create a HashMap to store element and its index.
2. Traverse the array.
3. For each element, check if (target - current) exists in map.
4. If yes, return the indices.
5. Otherwise, store the current element in the map.

@code.withnarayan

Java Code (Two Sum)

```java
import java.util.*;
public class TwoSum {
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i); // store element and index
        }
        return new int[]{}; // no solution
    }
}
```

@code.withnarayan

Dry Run

nums = [2, 7, 11, 15], target = 9

i	nums[i]	complement	map	Action
0	2	7	{}	store (2, 0)
1	7	2	{2:0}	found → return [0,1]
2	11	-	-	-
3	15	-	-	-

Complexity

@code.withnarayan

- ★ Time Complexity : O(n)
- ★ Space Complexity : O(n)

Important Questions (Practice)

1. Two Sum (Easy)
2. Contains Duplicate (Easy)
3. Valid Anagram (Easy)
4. Group Anagrams (Medium)
5. Subarray Sum Equals K (Medium)



2. Two Pointers Pattern

★ Use two pointers to solve problems efficiently, especially on arrays and strings ★

What is this Pattern ?

- ★ Use two pointers (left and right or same direction) to traverse the array or string.
- ★ Helps to reduce time complexity from $O(n^2)$ to $O(n)$.
- ★ Commonly used in sorted arrays, pairs, subarrays, and string problems.

When to use ?

- ☑ Array is sorted
- ☑ Find a pair with given sum
- ☑ Remove duplicates
- ☑ Check palindrome
- ☑ Find subarray / substring
- ☑ Many array and string problems

Types of Two Pointers

1. Opposite Pointers (left & right)
2. Same Direction Pointers (both move forward)
3. Fast & Slow Pointers (used in linked list/cycle)

@code.withnarayan @code.withnarayan

Example 1 : Pair Sum in Sorted Array

Given a sorted array, find if there exist two elements such that their sum is equal to target.

Input : nums = [2, 7, 11, 15], target = 9

Output : true (2 + 7 = 9)

Approach (Opposite Pointers)

1. Start with left = 0 and right = n - 1.
2. Calculate sum = nums[left] + nums[right].
3. If sum == target → return true.
4. If sum < target → move left++ (need bigger sum).
5. If sum > target → move right-- (need smaller sum).

Java Code (Pair Sum)

```java
public class PairSum {
    public static boolean hasPairSum(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left < right) {
            int sum = nums[left] + nums[right];
            if (sum == target) return true;
            else if (sum < target) left++;
            else right--;
        }
        return false;
}
```

@code.withnarayan

Dry Run

nums = [2, 7, 11, 15], target = 9

Step	left	right	nums[left]	nums[right]	sum	Action
1	0	3	2	15	17	sum > 9 right--
2	0	2	2	11	13	sum > 9 right--
3	0	1	2	7	9	sum == 9 return true

Complexity

@code.withnarayan

- ★ Time Complexity : $O(n)$
- ★ Space Complexity : $O(1)$

Other Common Problems using Two Pointers

1. Remove Duplicates from Sorted Array (Easy)
2. Move Zeroes (Easy)
3. Valid Palindrome (Easy)
4. 3 Sum (Medium)
5. Container With Most Water (Medium)

@code.withnarayan

Key Takeaways



- ☑ Two pointers reduce nested loops.
- ☑ Very useful for sorted arrays and strings.
- ☑ Can be applied in many problem variations.
- ☑ Identify the pointer movement condition.
- ☑ Practice multiple problems to master this pattern.

3. Sliding Window Pattern

★ Used to solve subarray / substring problems efficiently using a window (range). ★

What is Sliding Window ?

- ★ Maintain a window (range) using two pointers (left and right).
- ★ Efficiently solve subarray/substring problems in O(n) time.
- ★ Used for problems on arrays, strings and sequences.
- ★ Two types: Fixed Size Window and Variable Size Window.

When to use ?

- ☑ Find subarray/substring with sum, length, or count.
- ☑ Longest or shortest subarray/substring satisfying a condition.
- ☑ Maximum/minimum in a window.
- ☑ Problems with continuous elements.
- ☑ Replace nested loops ($O(n^2)$) with sliding window ($O(n)$).

Types of Sliding Window

1. **Fixed Size Window**
   (window size is constant)
   E.g., Maximum sum of subarray of size k.
2. **Variable Size Window**
   (window size changes based on condition)
   E.g., Longest substring without repeating characters.

Example 1 : Maximum Sum Subarray of Size K

Given an array and an integer k, find the maximum sum of any subarray of size k.

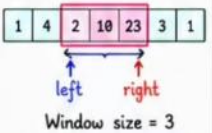
Input : nums = [1, 4, 2, 10, 23, 3, 1], k = 3

Output : 35 (subarray [2, 10, 23])

Dry Run

nums = [1, 4, 2, 10, 23, 3, 1], k = 3

Step	Window	Sum	Max Sum
1	[1, 4, 2]	7	7
2	[4, 2, 10]	16	16
3	[2, 10, 23]	35	35
4	[10, 23, 3]	36	36
5	[23, 3, 1]	27	36



Java Code (Fixed Size Window)

```java
public class MaxSumSubarray {
    public static int maxSum(int[] nums, int k) {
        int windowSum = 0, maxSum = 0;
        for (int i = 0; i < nums.length; i++) {
            windowSum += nums[i];
            if (i >= k - 1) {
                maxSum = Math.max(maxSum, windowSum);
                windowSum -= nums[i - k +1]; // slide window
            }
        }
        return maxSum;
    }
}
```

Java Code (Longest Substring)

```java
public class LongestSubstring {
    public static int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            map.put(ch, map.getOrDefault(ch, 0) + 1);
            while (map.get(ch) > 1) {
                char leftChar = s.charAt(left);
                map.put(leftChar, map.get(leftChar) - 1);
                left++;
            }
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

Example 2 : Longest Substring Without Repeating Characters

Given a string s, find the length of the longest substring without repeating characters.

Input : s = "abcabcbb"

Output : 3 ("abc")

Approach (Variable Size Window)

1. Use a HashMap/HashSet to store character frequency.
2. Move right pointer and add character.
3. If duplicate found, move left pointer until it is removed.
4. Track maximum length during the process.

Complexity

- ★ Time Complexity : O(n)
- ★ Space Complexity : O(min(n, k))



Important Questions (Practice)

1. Maximum Sum Subarray of Size K (Easy)
2. Longest Substring Without Repeating Characters (Medium)
3. Minimum Size Subarray Sum (Medium)
4. Fruit Into Baskets (Medium)
5. Longest Repeating Character Replacement (Medium)

4. Prefix Sum Pattern

★ Used to solve range sum, subarray sum and array update problems efficiently. ★

What is Prefix Sum ?

- ★ Prefix sum is an array where each element stores the sum of all previous elements (including itself).
- ★ Helps to find the sum of a subarray in O(1) time after preprocessing.
- ★ Commonly used in range queries, subarray sum equals K, count subarrays, and difference array problems.

When to use ?

- ☑ Find sum of elements in a range [l, r] quickly.
- ☑ Find subarray with given sum K.
- ☑ Count number of subarrays with given sum.
- ☑ Solve problems with range updates using difference array.
- ☑ Used in both arrays and matrices.

Prefix Sum Formula

```
prefix[i] = prefix[i - 1] + nums[i]
prefix[0] = nums[0]
```

Range Sum Query

```
sum(l, r) = prefix[r] - prefix[l - 1]
for l > 0
sum(0, r) = prefix[r]
```

Example 1 : Range Sum Query

Given an array nums, find the sum of elements between index l and r (inclusive).

Input : nums = [2, 4, 6, 8, 10], l = 1, r = 3

Output : 18 (4 + 6 + 8)

Prefix Array Construction

Index (i)	nums[i]	prefix[i]
0	2	2
1	4	6
2	6	12
3	8	20
4	10	30

Query : sum(1, 3)

= prefix[3] - prefix[0]

= 20 - 2

= 18

Java Code (Prefix Sum Array)

```java
public class PrefixSum {
  public static int[] buildPrefix(int[] nums) {
    int n = nums.length;
    int[] prefix = new int[n];
    prefix[0] = nums[0];
    for (int i = 1; i < n; i++) {
      prefix[i] = prefix[i - 1] + nums[i];
    }
    return prefix;
  }
}
```

Example 2 : Subarray Sum Equals K

Given an array nums and an integer k, find the number of subarrays whose sum equals k.

Input : nums = [1, 2, 3, 1, 2], k = 3

Output : 3 ([1,2], [3], [1,2])

Approach (Using Prefix Sum + HashMap)

1. Maintain a map <prefixSum, count>.
2. Initialize map with (0, 1).
3. Iterate through the array and compute prefix sum.
4. Check if (prefixSum - k) exists in map.
5. Add its count to the answer and update map.

Java Code (Subarray Sum Equals K)

```java
import java.util.*;
public class SubarraySumK {
  public static int countSubarraySum(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1);  // base case
    int prefixSum = 0, count = 0;
    for (int num : nums) {
      prefixSum += num;
      if (map.containsKey(prefixSum - k)) {
        count += map.get(prefixSum - k);
      }
      map.put(prefixSum, map.getOrDefault(prefixSum, 0) + 1);
    }
    return count;
  }
}
```

Dry Run

nums = [1, 2, 3, 1, 2], k = 3

i	nums[i]	prefixSum	prefixSum - k	map (key:count)	count
-	-	0	-	{0:1}	0
0	1	1	-2	{0:1, 1:1}	0
1	2	3	0	{0:1, 1:1, 3:1}	1
2	3	6	3	{0:1, 1:1, 3:1, 6:1}	2
3	1	7	4	{0:1, 1:1, 3:1, 6:1, 7:1}	2
4	2	9	6	{0:1, 1:1, 3:1, 6:1, 7:1, 9:1}	3

Complexity

★ Time Complexity : O(n)

★ Space Complexity : O(n)

Important Questions (Practice)

1. Range Sum Query (Easy)
2. Subarray Sum Equals K (Medium)
3. Count Subarrays with Sum Divisible by K (Medium)
4. Maximum Sum Subarray (Medium)
5. Continuous Subarray Sum (Medium)

5. Binary Search Pattern

★ Used to search an element or find an answer efficiently in a sorted array or search space. ★

What is Binary Search ?

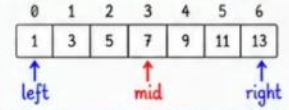
- ★ Binary search is a searching technique that works on a sorted array.
- ★ We repeatedly divide the search space into half.
- ★ Time Complexity is O(log n).
- ★ Used to find an element, first/last occurrence, lower/upper bound, or to search in an answer space.

When to use ?

- ☑ Array is sorted.
- ☑ Need to find an element quickly.
- ☑ Find first/last occurrence.
- ☑ Find lower/upper bound.
- ☑ Search in rotated sorted array.
- ☑ Binary search on answer (min/max).
- ☑ Problems where answer lies in a range.

Basic Idea

1. Find mid = left + (right - left) / 2
2. If arr[mid] == target → found
3. If arr[mid] < target → search right (left = mid + 1)
4. If arr[mid] > target → search left (right = mid - 1)



Example 1 : Search in Sorted Array

Given a sorted array nums and a target, return the index of target if found, otherwise -1.

Input : nums = [1, 3, 5, 7, 9, 11], target = 7

Output : 3

Explanation : mid = 2 → 5 < 7 → search right
mid = 4 → 9 > 7 → search left
mid = 3 → 7 found → return 3

Java Code (Search in Sorted Array)

```
public class BinarySearch {
    public static int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) left = mid + 1;
            else right = mid - 1;
        }
        return -1; // not found
    }
}
```

Dry Run

nums = [1, 3, 5, 7, 9, 11], target = 7

Step	left	right	mid	nums[mid]	Action
1	0	5	2	5	5 < 7 left = 3
2	3	5	4	9	9 > 7 right = 3
3	3	3	3	7	Found return 3

Example 2 : First and Last Occurrence

Given a sorted array nums and a target, find the first and last position of target. If not found, return [-1, -1].

Input : nums = [1, 2, 2, 2, 3, 4], target = 2

Output : [1, 3]

Java Code (First Occurrence)

```
public static int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1, ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid;
            right = mid - 1; // search left part
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}
```

Java Code (Last Occurrence)

```
public static int lastOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1, ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid;
            left = mid + 1; // search right part
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}
```

Example 3 : Search in Rotated Sorted Array

Given a rotated sorted array (no duplicates), find the index of target.

Input : nums = [4, 5, 6, 7, 0, 1, 2], target = 0

Output : 4



Approach (Rotated Array)

1. Find mid.
2. Check which half is sorted (left or right).
3. If target lies in sorted half → search in that half.
4. Otherwise search in the other half.

Complexity

- ★ Time Complexity : O(log n)
- ★ Space Complexity : O(1)

Important Questions (Practice)

1. Search in Sorted Array (Easy)
2. First and Last Occurrence (Medium)
3. Search in Rotated Sorted Array (Medium)
4. Find Minimum in Rotated Sorted Array (Medium)
5. Search in 2D Matrix (Medium)

6. Fast & Slow Pointers Pattern

★ Use fast and slow pointers to solve problems in linked lists or arrays (cycle detection, middle, etc.) ★

What is Fast & Slow Pointers ?

- ★ Use two pointers moving at different speeds (slow = 1 step, fast = 2 steps).
- ★ Helps to detect cycles, find middle element and solve many linked list problems efficiently.
- ★ Time Complexity is O(n).
- ★ Space Complexity is O(1).
- ★ Very common in coding interviews.

When to use ?

- ☑ Detect if a cycle exists in a linked list.
- ☑ Find the middle node of a linked list.
- ☑ Find the starting node of a cycle.
- ☑ Find the length of a cycle.
- ☑ Used in problems on arrays (like cycle in array).

@code.withnarayan

Pointer Movement

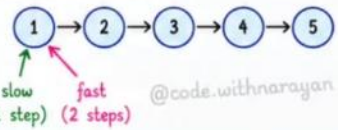
Slow Pointer

Moves 1 step at a time
(slow = slow.next)

Fast Pointer

Moves 2 steps at a time
(fast = fast.next.next)

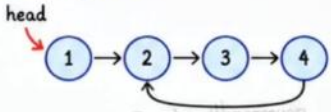
Visual Example



@code.withnarayan

Example 1 : Detect Cycle in Linked List

Given the head of a linked list, determine if there is a cycle in it.



@code.withnarayan

Java Code (Detect Cycle)

```java
public boolean hasCycle(ListNode head) {
    if (head == null) return false;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true; // cycle found
    }
    return false; // no cycle
}
```

@code.withnarayan

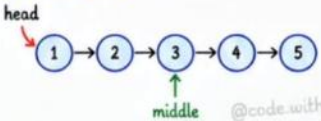
Dry Run (Detect Cycle)

Step	slow	fast	Condition	Action
1	1	1	-	move
2	2	3	slow != fast	
3	3	5	slow != fast	
4	4	3	slow == fast	Cycle found

@code.withnarayan

Example 2 : Find Middle of Linked List

Given the head of a linked list, return the middle node. If even number of nodes, return the second middle.



@code.withnarayan

Java Code (Find Middle)

```java
public ListNode middleNode(ListNode head) {
    if (head == null) return null;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow; // middle node
}
```

@code.withnarayan

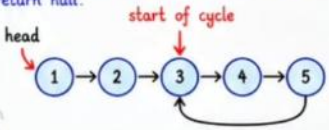
Complexity

- ★ Time Complexity : O(n)
- ★ Space Complexity : O(1)



Example 3 : Find Start of Cycle

Given the head of a linked list, return the node where the cycle begins. If no cycle, return null.



Java Code (Cycle Start)

```java
public ListNode detectCycle(ListNode head) {
    if (head == null) return null;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow.next;
                fast = fast.next;
            }
            return slow; // start of cycle
        }
    }
    return null; // no cycle
}
```

@code.withnarayan

Important Questions (Practice)

1. Linked List Cycle (Easy)
2. Find Middle of Linked List (Easy)
3. Linked List Cycle II (Medium)
4. Length of Cycle in Linked List (Medium)
5. Happy Number / Cycle in Array (Medium)

@code.withnarayan

7. Linked List Manipulation Pattern

★ Common operations on linked list like reverse, merge, delete, add, and find nodes using pointers. ★

What is this Pattern ?

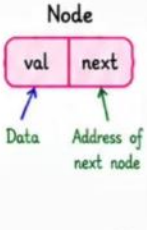
- ★ Used to manipulate nodes in a Linked List efficiently.
- ★ Common in coding interviews.
- ★ Problems include reverse, merge, delete, find middle, remove Nth node, detect cycle etc.
- ★ Usually solved using pointer manipulation and a dummy node.
- ★ Time Complexity is O(n).
- ★ Space Complexity is O(1).

When to use ?

- ☑ When question involves a singly or doubly linked list.
- ☑ Need to modify node connections (next pointers).
- ☑ Insert, delete or reverse nodes.
- ☑ Merge two sorted linked lists.
- ☑ Remove Nth node from end.
- ☑ Find middle node.
- ☑ Often used with fast & slow pointers and dummy node technique.

Basic Node Structure (Java)

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) {
      this.val = val;
      this.next = null;
    }
}
```

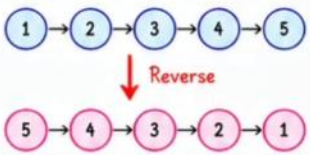


Example 1 : Reverse Linked List

Given the head of a linked list, reverse the list and return the new head.

Input : 1 → 2 → 3 → 4 → 5

Output : 5 → 4 → 3 → 2 → 1



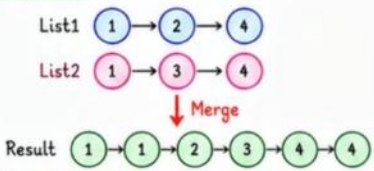
Example 2 : Merge Two Sorted Lists

Given heads of two sorted linked lists, merge them into one sorted linked list.

Input : List1: 1 → 2 → 4

List2: 1 → 3 → 4

Output : 1 → 1 → 2 → 3 → 4 → 4

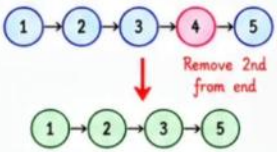


Example 3 : Remove Nth Node from End

Given the head of a linked list and an integer n, remove the Nth node from the end and return the head.

Input : 1 → 2 → 3 → 4 → 5, n = 2

Output : 1 → 2 → 3 → 5



Java Code (Reverse Linked List)

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Java Code (Merge Two Sorted Lists)

```
public ListNode mergeTwoLists(ListNode l1,
                              ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            curr.next = l1; l1 = l1.next;
        } else {
            curr.next = l2; l2 = l2.next;
        }
        curr = curr.next;
    }
    curr.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

Java Code (Remove Nth Node from End)

```
public ListNode removeNthFromEnd(ListNode head,
                                 int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i <= n; i++)
        fast = fast.next;
    while (fast != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;
    return dummy.next;
}
```

Dry Run (Remove 2nd from End)

List: 1 → 2 → 3 → 4 → 5, n = 2

Step	fast	slow	Action
1	3	0	Move fast 3 steps
2	5	3	Move both until fast = null
3	null	4	Delete slow.next (4)

Complexity

- ★ Time Complexity : O(n)
- ★ Space Complexity : O(1)



Key Takeaways

- ☑ Use dummy node to handle edge cases.
- ☑ Pointer manipulation is the key.
- ☑ Works for many linked list problems.
- ☑ Practice variations to master this pattern.

Important Questions (Practice)

1. Reverse Linked List (Easy)
2. Merge Two Sorted Lists (Easy)
3. Remove Nth Node from End (Medium)
4. Reorder List (Medium)
5. Add Two Numbers (Medium)

@code.withnarayan

8. Stack & Monotonic Stack Pattern

★ Used to solve problems involving brackets, next greater/smaller element, and monotonic sequences. ★

What is this Pattern ?

- ★ Stack follows LIFO (Last In First Out) principle.
- ★ Used to solve problems with nested structures, next greater/smaller element, and maintaining order.
- ★ Monotonic Stack maintains elements in increasing or decreasing order.
- ★ Time Complexity is O(n).
- ★ Space Complexity is O(n).

When to use ?

- ☑ When problem involves parentheses/brackets.
- ☑ When we need to find next greater/smaller element.
- ☑ When we need to maintain monotonic increasing/decreasing order.
- ☑ When solving histogram, stock span, daily temperatures etc.
- ☑ When we need to keep track of elements in a specific order.

Types of Stack Problems

1. Bracket / Parentheses Problems (Valid Parentheses, Balanced Brackets)
2. Next Greater/Smaller Element (NGE, NSE, Circular NGE)
3. Monotonic Stack (Maintain increasing or decreasing order)
4. Histogram / Area Problems (Largest Rectangle in Histogram)
5. Stock Span / Daily Temperatures (Next greater element variations)

@code.withnarayan

Example 1 : Valid Parentheses

Given a string s containing '(', ')', '{', '}', '[', ']', check if the parentheses are valid.

Input : s = "()[]{}"

Output : true

Approach :

1. If opening bracket → push to stack.
2. If closing bracket → check top and pop.
3. If stack empty at end → valid.

Java Code

```
public class ValidParentheses {
    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == [[') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c == ')' && top != '(') ||
                    (c == '}' && top != '{') ||
                    (c == ']' && top != '['))
                    return false;
            }
        }
        return stack.isEmpty();
    }
}
```

@code.withnarayan

Example 2 : Next Greater Element

Given an array nums, find the next greater element for each element. If no greater element, return -1.

Input : nums = [4, 5, 2, 25]

Output : [5, 25, 25, -1]

Approach : (Monotonic Decreasing Stack)

1. Traverse from right to left.
2. Maintain a stack of greater elements.
3. Pop smaller elements and set result.

Java Code

```
public class NextGreaterElement {
    public static int[] nextGreater(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        Stack<Integrer> stack = new Stack<>();
        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() &&
                   ttack.peek() <= nums[i]) {
                stack.pop();
            }
            ans[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums[i]);
        }
        return ans;
    }
}
```

@code.withnarayan

Example 3 : Daily Temperatures

Given an array of temperatures, return an array where each element represents the number of days you have to wait for a warmer temperature.

Input : temps = [73, 74, 75, 71, 69, 72, 76, 73]

Output : [1, 1, 4, 2, 1, 1, 0, 0]

Approach : (Monotonic Decreasing Stack)

1. Traverse from left to right.
2. Maintain stack of indices with decreasing temperatures.
3. When a warmer temperature is found, update answer for indices in stack.

Java Code

```
public class DailyTemperatures {
    pubic static int[] dailyTemperatures(int[] temps) {
        int n = temps.length;
        int[] ans = new int[n];
        Stack<Integer> stack = new Stack<>();
        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() &&
                   temps[i] > temps[stack.peek()]) {
                int idx = stack.pop();
                ans[idx] = i - idx;
            }
            stack.push(i);
        }
        return ans;
    }
} }
```

@code.withnarayan

Dry Run (Next Greater Element)

nums = [4, 5, 2, 25]

i	nums[i]	Stack (top right)	ans
3	25	[]	[-1]
2	2	[25]	[-1, 25]
1	5	[25]	[-1, 25, 25]
0	4	[5, 25]	[5, 25, 25, -1]

@code.withnarayan

Monotonic Stack Concept

Monotonic Increasing Stack (for Next Smaller Element)



Monotonic Decreasing Stack (for Next Greater Element)



@code.withnarayan

Complexity



- ★ Time Complexity : O(n)
- ★ Space Complexity : O(n)

Important Questions (Practice)

1. Valid Parentheses (Easy)
2. Next Greater Element (Medium)
3. Next Smaller Element (Medium)
4. Largest Rectangle in Histogram (Hard)
5. Daily Temperatures (Medium)

9. Binary Tree DFS & BFS Pattern

★ Used to traverse a binary tree using Depth First Search (Preorder, Inorder, Postorder) and Breadth First Search (Level Order). ★

What is Tree Traversal ?

- ★ Traversal means visiting all nodes of a binary tree in a specific order.
- ★ DFS uses recursion or stack (Preorder, Inorder, Postorder).
- ★ BFS uses a queue (Level Order).
- ★ Time Complexity for all traversals is O(n).
- ★ Space Complexity is O(h) for DFS (h = height) and O(n) for BFS (worst case for queue).

@code.withnarayan

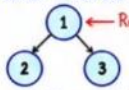
When to use ?

- ☑ When we need to process or print all nodes of a binary tree.
- ☑ Used in many tree problems like sum, max, search, path etc.
- ☑ Preorder is useful to create a copy of tree.
- ☑ Inorder is useful for BST (sorted order).
- ☑ Postorder is useful for deleting a tree.
- ☑ Level Order is useful for level wise processing.

Types of Traversal

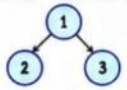
@code.withnarayan

1. Preorder Traversal (Root → Left → Right)



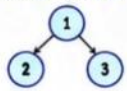
Order : 1 2 3

2. Inorder Traversal (Left → Root → Right)



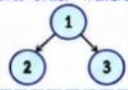
Order : 2 1 3

3. Postorder Traversal (Left → Right → Root)



Order : 2 3 1

4. Level Order Traversal (BFS) (Level by Level)

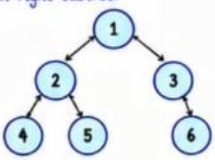


Order : 1 2 3

@code.withnarayan

Example 1 : Preorder Traversal

Visit root first, then left subtree, then right subtree.



Preorder : 1 2 4 5 3 6

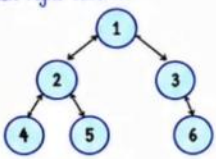
Java Code (Preorder)

```
public void preorder(TreeNode root,
        List<Integer> ans) {
    if (root == null) return;
    ans.add(root.val);    // root
    preorder(root.left, ans);  // left
    preorder(root.right, ans); // right
}
```

@code.withnarayan

Example 2 : Inorder Traversal

Visit left subtree, then root, then right subtree.



Inorder : 4 2 5 1 3 6

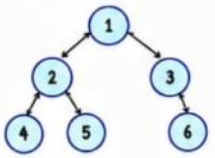
Java Code (Inorder)

```
public void inorder(TreeNode root,
        List<Integer> ans) {
    if (root == null) return;
    inorder(root.left, ans);  // left
    ans.add(root.val);        // root
    inorder(root.right, ans); // right
}
```

@code.withnarayan

Example 3 : Postorder Traversal

Visit left subtree, then right subtree, then root.



Postorder : 4 5 2 6 3 1

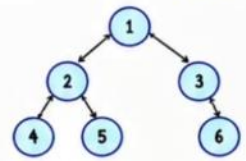
Java Code (Postorder)

```
public void postorder(TreeNode root,
        List<Integer> ans) {
    if (root == null) return;
    postorder(root.left, ans);  // left
    postorder(root.right, ans); // right
    ans.add(root.val);          // root
}
```

@code.withnarayan

Example 4 : Level Order Traversal

Visit nodes level by level using a queue.



Level Order : 1 2 3 4 5 6

Java Code (Level Order)

```
public List<List<Integer>> levelOrder(
        TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        res.add(level);
    }
    return res;
}
```

@code.withnarayan

Dry Run (Level Order)

Step	Queue	Node	Result (Level)
1	[1]	1	[1]
2	[2, 3]	2	[1, 2]
3	[3, 4, 5]	3	[1, 2, 3]
4	[4, 5, 6]	4	[1, 2, 3, 4]
5	[5, 6]	5	[1, 2, 3, 4, 5]
6	[6]	6	[1, 2, 3, 4, 5, 6]

@code.withnarayan

Complexity



- ★ Time Complexity : O(n)
- ★ Space Complexity :
  - DFS : O(h) (h = height)
  - BFS : O(n) (worst case)

Key Takeaways

- ☑ Preorder : useful for tree copy/serialization.
- ☑ Inorder : gives sorted order for BST.
- ☑ Postorder : useful for deleting tree.
- ☑ Level Order : used for level wise problems.

Important Questions (Practice)

1. Binary Tree Inorder Traversal (Easy)
2. Binary Tree Level Order Traversal (Easy)
3. Maximum Depth of Binary Tree (Easy)
4. Same Tree (Easy)
5. Binary Tree Zigzag Level Order Traversal (Medium)

10. Heap & Top K Elements Pattern

★ Used to find largest/smallest elements, Kth element, top K frequent elements and solve many optimization problems. ★

What is a Heap ?

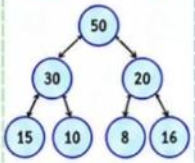
- ★ A heap is a complete binary tree that satisfies heap property.
- ★ Two types :
  - Max Heap : parent ≥ children
  - Min Heap : parent ≤ children
- ★ Usually implemented using PriorityQueue in Java.
- ★ Time Complexity for insert/delete : O(log n).
- ★ Used to efficiently find min/max element or top K elements.

When to use ?

- ☑ When we need Kth largest/smallest element.
- ☑ When we need top K frequent elements.
- ☑ When we need to maintain a sorted order of elements.
- ☑ When we need to process a stream of data.
- ☑ Used in problems like median, scheduling, merging K lists etc.
- ☑ Helps to efficiently handle large data with log n operations.

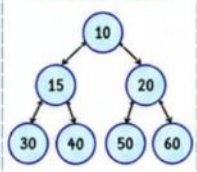
Types of Heap

Max Heap



- ★ Parent ≥ Children
- ★ Largest element at root
- ★ Used to find maximum element

Min Heap



- ★ Parent ≤ Children
- ★ Smallest element at root
- ★ Used to find minimum element

@code.withnarayan

Example 1 : Kth Largest Element

Given an array nums and an integer k, return the kth largest element.

Input : nums = [3, 2, 1, 5, 6, 4], k = 2

Output : 5

Approach : (Min Heap of size k)

1. Add elements to min heap.
2. If size > k, remove smallest.
3. Top of heap is kth largest.

@code.withnarayan

Example 2 : Top K Frequent Elements

Given an array nums and an integer k, return the k most frequent elements.

Input : nums = [1, 1, 1, 2, 2, 3], k = 2

Output : [1, 2]

Approach : (Min Heap + Frequency Map)

1. Count frequency using HashMap.
2. Add (freq, num) to min heap.
3. Keep heap size ≤ k.
4. Extract elements from heap.

@code.withnarayan

Example 3 : Kth Smallest Element

Given an array nums and an integer k, return the kth smallest element.

Input : nums = [7, 10, 4, 3, 20, 15], k = 3

Output : 7

Approach : (Max Heap of size k)

1. Add elements to max heap.
2. If size > k, remove largest.
3. Top of heap is kth smallest.

@code.withnarayan

Java Code (Kth Largest Element)

```java
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap =
        new PriorityQueue<>(); // min heap
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll(); // remove smallest
        }
    }
    return minHeap.peek(); // kth largest
}
```

@code.withnarayan

Java Code (Top K Frequent Elements)

```java
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) {
        freq.put(num, freq.getOrDefault(num, 0) + 1);
    PriorityQueue<int[]> minHeap =
        new PriorityQueue<>((a, b) -> a[0] - b[0]);
    for (Map.Entry<Integer, Integer> entry :
        freq.entrySet() {
        minHeap.offer(new int[]{entry.getValue(),
                                entry.getKey()});
        if (minHeap.size() > k) minHeap.poll();
    }
    int[] res = new int[k];
    for (int i = k - 1; i >= 0; i--)
        res[i] = minHeap.poll()[1];
    return res;
}
```

@code.withnarayan

Java Code (Kth Smallest Element)

```java
public int findKthSmallest(int[] nums, int k) {
    PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>(Collections.reverseOrder());
    for (int num : nums) {
        maxHeap.offer(num);
        if (maxHeap.size() > k) {
            maxHeap.poll(); // remove largest
        }
    }
    return maxHeap.peek(); // kth smallest
}
```

@code.withnarayan

Dry Run (Kth Largest, k = 2)

Step	num	Heap (min heap, size ≤ 2)
1	3	[3]
2	2	[2, 3]
3	1	[2, 3] (1 removed)
4	5	[3, 5]
5	6	[5, 6] (3 removed)
6	4	[5, 6] (4 removed)

Result : Top = 5 (2nd largest)

@code.withnarayan

Complexity



- ★ Time Complexity : O(n log k)
- ★ Space Complexity : O(k)

Key Takeaways

- ☑ Use PriorityQueue in Java.
- ☑ Min heap for Kth largest / top K.
- ☑ Max heap for Kth smallest.
- ☑ Time complexity is O(n log k).
- ☑ Useful in many real-world and interview problems.

Important Questions (Practice)

1. Kth Largest Element in an Array (Easy)
2. Kth Smallest Element in an Array (Medium)
3. Top K Frequent Elements (Medium)
4. Find Median from Data Stream (Hard)
5. Merge K Sorted Lists (Hard)

@code.withnarayan

11. Graph Traversal (BFS & DFS) Pattern

★ Used to traverse or explore all nodes in a graph using Breadth First Search (BFS) and Depth First Search (DFS). ★

What is a Graph ?

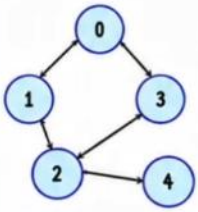
- ★ A graph is a non-linear data structure consisting of nodes (vertices) and edges.
- ★ Types :
  - Directed Graph (edges have direction)
  - Undirected Graph (edges have no direction)
- ★ Graphs can be represented using Adjacency List or Adjacency Matrix.
- ★ Used in many real-world problems like networking, social media, maps, path finding etc.

@code.withnarayan

When to use ?

- ☑ When we need to visit all nodes in a graph.
- ☑ When we need shortest path (unweighted) → use BFS.
- ☑ When we need to explore all possible paths → use DFS.
- ☑ When working with connected components, cycles, topological sorting etc.
- ☑ Used in problems related to networks, routing, dependency resolution, and more.

Graph Representation (Adjacency List)



Undirected Graph

Adjacency List

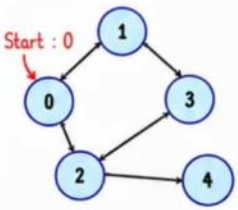
0	[1, 2]
1	[0, 3]
2	[0, 3, 4]
3	[1, 2]
4	[2]

@code.withnarayan

Example 1 : Breadth First Search (BFS)

Visit all nodes level by level using a queue.

Traversal Order : 0 1 2 3 4



BFS : 0 → 1 → 2 → 3 → 4

Java Code (BFS)

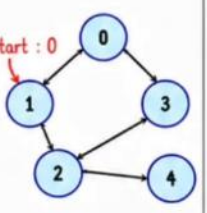
```
public void bfs(int start, int V, List<List<Integer>> adj) {
    boolean[] visited = new boolean[V];
    Queue<Integer> q = new LinkedList<>();
    visited[start] = true;
    q.offer(start);
    while (!q.isEmpty()) {
        int node = q.poll();
        System.out.print(node + " ");
        for (int nei : adj.get(node)) {
            if (!visited[nei]) {
                visited[nei] = true;
                q.offer(nei);
            }
        }
    }
}
```

@code.withnarayan

Example 2 : Depth First Search (DFS)

Visit all nodes using recursion (or stack).

Traversal Order : 0 1 2 3 4



DFS : 0 → 1 → 3 → 2 → 4

Java Code (DFS - Recursive)

```
public void dfs(int node, boolean[] visited,
                List<List<Integer>> adj) {
    visited[node] = true;
    System.out.print(node + " ");
    for (int nei : adj.get(node)) {
        if (!visited[nei]) {
            dfs(nei, visited, adj);
        }
    }
}
public void dfsTraversal(int start, int V,
                List<List<Integer>> adj) {
    boolean[] visited = new boolean[V];
    dfs(start, visited, adj);
}
```

@code.withnarayan

Dry Run (BFS from 0)

Step	Queue	Visited	Action
1	[0]	{0}	start from 0
2	[1, 2]	{0,1,2}	visit 0, add 1,2
3	[2, 3]	{0,1,2,3}	visit 1, add 3
4	[3, 4]	{0,1,2,3,4}	visit 2, add 4
5	[4]	{0,1,2,3,4}	visit 3 (no new)
6	[]	{0,1,2,3,4}	visit 4 (no new)

BFS Traversal : 0 1 2 3 4

@code.withnarayan

Time & Space Complexity



- ★ BFS : Time → O(V + E)
  Space → O(V)
- ★ DFS : Time → O(V + E)
  Space → O(V)

V = number of vertices (nodes)
E = number of edges

Applications

- ★ Shortest path in unweighted graph (Social Networks, Maps)
- ★ Detecting connected components
- ★ Cycle detection
- ★ Topological sorting (DAG)
- ★ Finding all possible paths
- ★ Network routing and web crawling

Key Takeaways

- ☑ BFS uses a queue, DFS uses recursion/stack.
- ☑ BFS gives shortest path in unweighted graph.
- ☑ Both have time complexity O(V + E).
- ☑ Graph is represented using adjacency list.
- ☑ Very important for many real-world problems and coding interviews.

Important Questions (Practice)

1. Number of Connected Components (Easy)
2. Is Graph Bipartite? (Medium)
3. Detect Cycle in Undirected Graph (Medium)
4. Course Schedule (Topological Sort) (Medium)
5. Shortest Path in Unweighted Graph (Medium)

@code.withnarayan

12. Dynamic Programming (DP) Pattern

★ Technique to solve problems by breaking them into smaller overlapping subproblems and storing their results. ★

What is Dynamic Programming ?

- ★ DP solves problems with:
  - Overlapping subproblems
  - Optimal substructure
- ★ We store the result of subproblems to avoid repeated calculations.
- ★ Two approaches:
  - Top-Down (Memoization)
  - Bottom-Up (Tabulation)
- ★ Used in optimization problems.
- ★ Common in sequence, path, partition, and counting problems.

When to use DP ?

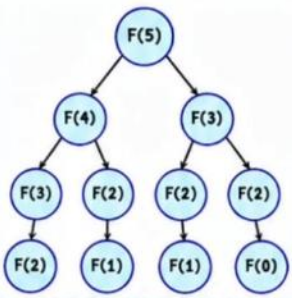
- ☑ When problem can be divided into smaller subproblems.
- ☑ When subproblems overlap (repeatedly solved).
- ☑ When optimal solution can be built from optimal solutions of subproblems.
- ☑ To find minimum/maximum value or count of ways.
- ☑ Common in arrays, strings, grids, sequences, partitions, knapsack problems etc.

Types of DP (with Example Problems)

1. Linear DP → Fibonacci, Climbing Stairs
2. 0/1 Knapsack → Subset Sum, Partition
3. Unbounded Knapsack → Coin Change
4. Grid DP → Unique Paths, Min Path Sum
5. String DP → Longest Common Subsequence, Edit Distance
6. DP on Subsequences → LIS (Longest Increasing Subsequence)
7. DP on Intervals → Matrix Chain Multiplication
8. DP on Bits (Bitmask) → TSP, Subset Problems

Example 1 : Fibonacci (Top-Down with Memoization)

$F(n) = F(n-1) + F(n-2)$



Java Code (Memoization)

```
public int fib(int n, int[] dp) {
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    dp[n] = fib(n - 1, dp) + fib(n - 2, dp);
    return dp[n];
}
// call
int n = 5;
int[] dp = new int[n + 1];
Arrays.fill(dp, -1);
System.out.println(fib(n, dp));
```

Example 2 : 0/1 Knapsack (Bottom-Up)

Find maximum value with given weight capacity.

Items

Item	Weight	Value
1	1	1
2	3	4
3	4	5

Capacity (W) = 4

Java Code (Tabulation)

```
public int knapsack(int[] wt, int[] val, int W) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (wt[i - 1] <= w) {
                dp[i][w] = Math.max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w]
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][W];
}
```

Example 3 : Longest Common Subsequence (LCS)

Find length of longest subsequence present in both strings.

S1 = "abcde" , S2 = "ace"

	0	a	c	e
0	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

LCS length = 3
(LCS = "ace")

Java Code (Tabulation)

```
public int lcs(String s1, String s2) {
    int n = s1.length(), m = s2.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1.charAt(i - 1) == s2.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

Time & Space Complexity

- ★ Memoization : Time → O(n), Space → O(n)
- ★ Tabulation : Time → O(n), Space → O(n)

(for most linear DP problems)

Key Takeaways

- ☑ Identify states and define recurrence relation.
- ☑ Use memoization (top-down) or tabulation (bottom-up).
- ☑ DP eliminates repeated calculations.
- ☑ Useful for optimization and counting problems.
- ☑ Widely used in coding interviews and competitive programming.

Important Questions (Practice)

1. Fibonacci Number (Easy)
2. Climbing Stairs (Easy)
3. 0/1 Knapsack (Medium)
4. Coin Change (Medium)
5. Longest Common Subsequence (Medium)

@code.withnarayan

13. Bit Manipulation Pattern

★ Bit manipulation means performing operations directly on the binary representation of numbers. ★

1. Basics of Bits

- ★ Computers store numbers in binary (0 and 1).
- ★ Each bit represents a power of 2.
- ★ We can use bitwise operators to manipulate bits.

@code.withnarayan

Position (i)	4	3	2	1	0
Value (2^i)	16	8	4	2	1

2. Common Bitwise Operators

Operator	Name	Example (a = 5, b = 3)	Result
&	AND	5 & 3 (0101 & 0011)	1
|	OR	5 | 3 (0101 | 0011)	7
^	XOR	5 ^ 3 (0101 ^ 0011)	6
~	NOT	~5 (000...0101)	-6
<<	Left Shift	5 << 1 (0101 << 1)	10
>>	Right Shift	5 >> 1 (0101 >> 1)	2
>>>	Unsigned Right Shift	5 >>> 1	2

3. Useful Bit Properties

- ★ X ^ X = 0
- ★ X ^ 0 = X
- ★ XOR is commutative and associative
- ★ n & (n - 1) removes the rightmost set bit
- ★ n & -n gives the rightmost set bit
- ★ n % 2 == 1 → last bit is 1 (odd)
- ★ n >> 1 is same as n / 2
- ★ n << 1 is same as n * 2

@code.withnarayan

4. Check if a Number is Even or Odd

- ★ If (n & 1) == 0 → Even
- ★ If (n & 1) == 1 → Odd

Java Code

```
public static boolean isEven(int n) {
    return (n & 1) == 0;
}
```

n = 5 → 0101
5 & 1 = 1
→ Odd

n = 4 → 0100
4 & 1 = 0
→ Even

@code.withnarayan

5. Check, Set, Clear and Toggle a Bit

- ★ Check i-th bit : (n & (1 << i)) != 0
- ★ Set i-th bit : n | (1 << i)
- ★ Clear i-th bit : n & ~(1 << i)
- ★ Toggle i-th bit : n ^ (1 << i)

Java Code

```
int n = 5;  // 0101
int i = 1;
System.out.println((n & (1 << i)) != 0); // check
System.out.println (n | (1 << i));   // set
System.out.println (n & ~(1 << i)); // clear
System.out.println (n ^ (1 << i));   // toggle
```

@code.withnarayan

6. Count Set Bits (Hamming Weight)

- ★ Method 1: Check each bit (O(32))
- ★ Method 2 : n & (n - 1) trick (O(k))

Java Code (Efficient)

```
public static int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        n = n & (n - 1); // remove rightmost set bit
        count++;
    }
    return count;
}
```

@code.withnarayan

7. Check if a Number is Power of Two

- ★ A number is power of two if only one bit is set.
- ★ Condition: n > 0 && (n & (n - 1)) == 0

Java Code

```
public static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

n = 8 → 1000
8 & 7 = 0
→ Power of Two

n = 10 → 1010
10 & 9 = 8 ≠ 0
→ Not a Power of Two

@code.withnarayan

8. Find the Rightmost Set Bit

- ★ n & -n gives the rightmost set bit.
- ★ Useful in many problems (subsets, bit masks).

Example:

n = 12 → 1100
-n = 0100
n & -n = 0100 = 4

9. Clear the Rightmost Set Bit

- ★ n & (n - 1) removes the rightmost set bit.

Example:

n = 12 → 1100
n - 1 = 1011
n & (n - 1) = 1000 = 8

10. Subset Generation using Bits

- ★ For a set of n elements, there are 2^n subsets.
- ★ We can use bit masking to generate all subsets.

Java Code

```
public static void printSubsets(int[] arr) {
    int n = arr.length;
    for (int mask = 0; mask < (1 << n); mask++) {
        System.out.print("[");
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                System.out.print(arr[i] + " ");
            }
        System.out.println("]");
    }
}
```

Example: arr = [1, 2]

mask (binary)	→	subset
00	→	{}
01	→	{1}
10	→	{2}
11	→	{1, 2}

@code.withnarayan

11. Common Questions

1. Count Set Bits
2. Power of Two
3. Single Number (XOR)
4. Find Missing Number
5. Subset Generation
6. Reverse Bits
7. Find Two Unique Numbers
8. Bitwise AND of Range

12. Time and Space Complexity

Problem	Time	Space
Check Even/Odd	O(1)	O(1)
Set/Clear/Toggle Bit	O(1)	O(1)
Count Set Bits	O(k) (k = set bits)	O(1)
Power of Two	O(1)	O(1)
Subset Generation	O(2^n)	O(1)

@code.withnarayan

Key Takeaways

- ★ Bit manipulation is fast and memory efficient.
- ★ Know common bit tricks and properties.
- ★ Useful in competitive programming and coding interviews.
- ★ Commonly used in: arrays, searching, hashing, dynamic programming, graphs (bit masks) and optimization problems.
- ★ Mastering bit manipulation helps in solving complex problems with simple logic.

14. Backtracking Pattern

★ Backtracking is a technique to explore all possible solutions by trying choices and undoing them (backtrack) when choice does not lead to a valid solution. ★

1. What is Backtracking ?

- ★ Solve problems by building solutions step by step.
- ★ If current choice is not valid, we backtrack (undo the choice).
- ★ Used for problems with multiple possible solutions (combinatorial).
- ★ Common in permutations, combinations, subsets, N-Queens, Sudoku etc.

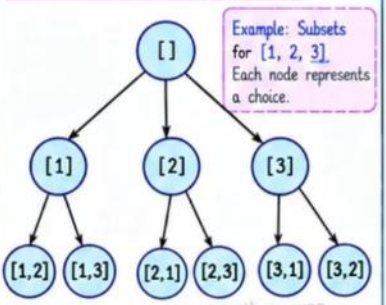
@code.withnarayan

2. General Template

Java Code (Template)

```
void backtrack(...) {
    // base condition
    if (isSolution) {
        // store or print result
        return;
    }
    for (each choice) {
        // make choice
        backtrack(...); // explore
        // undo choice (backtrack)
    }
}
```

3. Recursive Tree (Concept)



@code.withnarayan

4. Subsets (Power Set)

- ★ Find all possible subsets of a set.
- ★ For n elements, total subsets = 2^n.
- ★ Each element has two choices: Include or Exclude.

Output: [ ] , [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]

Example: nums = [1, 2, 3]

@code.withnarayan

Java Code

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(0, nums, new ArrayList<>(), res);
    return res;
}

void backtrack(int idx, int[] nums, List<Integer> cur,
               List<List<Integer>> res) {
    if (idx == nums.length) {
        res.add(new ArrayList<> (cur));
        return;
    }
    // include
    cur.add(nums[idx]);
    backtrack(idx + 1, nums, cur, res);
    cur.remove(cur.size() - 1);
    // exclude
    backtrack(idx + 1, nums, cur, res);
}
```

@code.withnarayan

5. Permutations

Output: [ ] , [1], [2], [3], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]

- ★ Find all possible arrangements of elements.
- ★ For n elements, total permutations = n!.
- ★ We choose each element and explore remaining elements.

Example: nums = [1, 2, 3]

@code.withnarayan

Java Code

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    boolean[] used = new boolean[nums.length];
    backtrack(nums, used, new ArrayList<>(), res);
    return res;
}
void backtrack(int[] nums, boolean[] used,
    List<Integer> cur, List<List<Integer>> res) {
    if (cur.size() == nums.length) {
        res.add(new ArrayList<>(cur));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        cur.add(nums[i]);
        backtrack(nums, used, cur, res);
        cur.remove(cur.size() - 1);
    }   used[i] = false;
}
```

6. Combinations

Output: [1,2], [1,3], [1,4], [2,3], [2,4], [3,4]

- ★ Find all combinations of k elements from n elements.
- ★ Order does not matter.
- ★ Total combinations = $^nC_k = \frac{n!}{k!(n-k)!}$

Example: n = 4, k = 2

@code.withnarayan

Java Code

```
public List<List<Integer>> combine(int n, int k) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(1, n, k, new ArrayList<>(), res);
    return res;
}

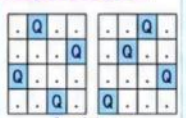
void backtrack(int start, int n, int k,
    List<Integer> cur, List<List<Integer>> res) {
    if (cur.size() == k) {
        res.add(new ArrayList<>(cur));
        return;
    }
    for (int i = start; i <= n; i++) {
        cur.add(i);
        backtrack(i + 1, n, k, cur, res);
        cur.remove(cur.size() - 1);
    }
}
```

@code.withnarayan

7. N-Queens Problem

- ★ Place N queens on an N × N chessboard so that no two queens attack each other.
- ★ Constraints: same row, column or diagonal.
- ★ Total solutions for N = 4 → 2

Output for N = 4:



2 solutions

Java Code (Simplified)

```
boolean isSafe(int row, int col, char[[]] board, int n) {
  for (int i = 0; i < row; i++) if (board[i][col] == 'Q') return false;
  for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
      if (board[i][j] == 'Q') return false;
  for (int i = row, j = col; i >= 0 && j < n; i--, j++)
      if (board[i][j] == 'Q') return false;
  return true;
}
```

@code.withnarayan

8. Other Backtracking Problems

1. Subsets (with duplicates)
2. Permutations (with duplicates)
3. Combination Sum
4. Combination Sum II
5. Palindrome Partitioning
6. N-Queens
7. Sudoku Solver
8. Word Search

9. Time & Space Complexity

Problem	Time Complexity	Space Complexity
Subsets	$O(2^n)$	$O(n)$
Permutations	$O(n!)$	$O(n)$
Combinations	$O(^nC_k)$	$O(k)$
N-Queens	$O(N!)$	$O(N)$
Sudoku Solver	$O(9^{N^2})$	$O(N^2)$

Key Takeaways

@code.withnarayan

- ★ Try all possible choices and backtrack when needed.
- ★ Use recursion and undo the choice.
- ★ Commonly used in combinatorial and constraint-based problems.
- ★ Important for coding interviews and competitive programming.

15. Intervals & Greedy Pattern

★ Greedy algorithm makes the locally optimal choice at each step with the hope of finding the globally optimal solution. ★

1. What are Intervals ?

- ★ An interval is a range [start, end].
- ★ Can be open/closed, but in problems we use closed intervals [s, e].
- ★ Common operations:
  - Check overlap
  - Merge overlapping intervals
  - Find non-overlapping intervals
  - Schedule maximum activities.

@code.withnarayan

2. What is Greedy Algorithm ?

- ★ Make the best local choice at each step without worrying about future choices.
- ★ Works when the problem has Greedy Choice Property and Optimal Substructure.

Examples:

- ★ Activity Selection
- ★ Interval Scheduling
- ★ Minimum Number of Platforms
- ★ Huffman Coding
- ★ Fractional Knapsack

@code.withnarayan

3. When to use Greedy ?

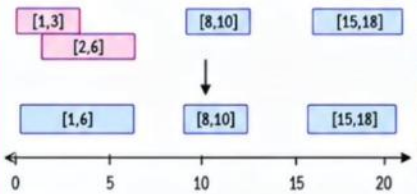
- ☑ When making a local optimal choice leads to global optimal solution.
- ☑ When problem has optimal substructure.
- ☑ Usually used in scheduling, intervals, mathematical optimization problems.
- ☑ Not always works (e.g., 0/1 Knapsack needs DP, not Greedy).

@code.withnarayan

4. Merge Intervals

★ Given a list of intervals, merge all overlapping intervals.

Example: intervals = [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]



Java Code

@code.withnarayan

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> res = new ArrayList<>();
    for (int[] in : intervals) {
        if (res.isEmpty() || res.get(res.size()-1)[1] < in[0]) {
            res.add(in);
        } else {
            res.get(res.size()-1)[1] = Math.max(res.get(res.size()-1)[1],
            in[1]);
        }
    }
    return res.toArray(new int[res.size()][]);
}
```

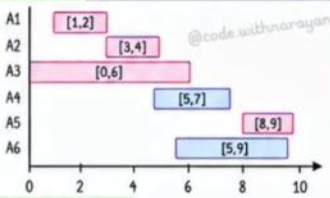
@code.withnarayan

Time Complexity: O(n log n)
Space Complexity: O(n)

5. Activity Selection Problem

★ Given start and end time of activities, select maximum number of non-overlapping activities.

Example: start = [1, 3, 0, 5, 8, 5]
end = [2, 4, 6, 7, 9, 9]
Output: 4 (activities: [1,2], [3,4], [5,7], [8,9])



@code.withnarayan

Java Code

```
static class Activity {
    int start, end;
    Activity(int s, int e) { start = s; end = e; }
}

public int maxActivities(Activity[] arr) {
    Arrays.sort(arr, (a, b) -> a.end - b.end);
    int count = 1, lastEnd = arr[0].end;
    for (int i = 1; i < arr.length; i++) {
        if (arr[i].start >= lastEnd) {
            count++;
            lastEnd = arr[i].end;
        }
    }
    return count;
}
```

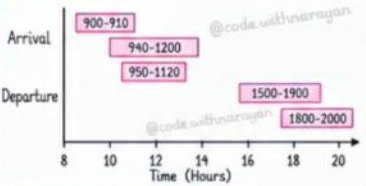
@code.withnarayan

Time Complexity: O(n log n)
Space Complexity: O(1)

6. Minimum Number of Platforms

★ Given arrival and departure time of trains, find minimum number of platforms required so that no train has to wait.

Example:
arrival = [900, 940, 950, 1100, 1500, 1800]
departure = [910, 1200, 1120, 1130, 1900, 2000]
Output: 3



@code.withnarayan

Java Code (Two Pointer)

```
public int findPlatform(int[] arr, int[] dep) {
    Arrays.sort(arr);
    Arrays.sort(dep);
    int i = 0, j = 0, platforms = 0, maxP = 0;
    while (i < arr.length && j < dep.length) {
        if (arr[i] <= dep[j]) {
            platforms++;
            maxP = Math.max(maxP, platforms);
            i++;
        } else {
            platforms--;
            j++;
        }
    }
    return maxP;
}
```

@code.withnarayan

Time Complexity: O(n log n)
Space Complexity: O(1)

7. Other Interval Problems

1. Insert Interval
2. Non-overlapping Intervals
3. Meeting Rooms (Can Attend All Meetings?)
4. Meeting Rooms II (Minimum Rooms)
5. Maximum Number of Non-overlapping Intervals
6. Interval List Intersections
7. Merge k Sorted Interval Lists
8. Task Scheduling (with cooldown)

8. Key Greedy + Interval Intuition

- ★ Sort intervals (usually by start time or end time).
- ★ Make locally optimal choice (e.g., pick earliest ending interval).
- ★ Use two pointers or sorting.
- ★ Always check for overlap: current.start <= previous.end
- ★ For maximum non-overlapping, pick interval with smallest end time.
- ★ Greedy works well for scheduling and interval problems.

9. Time & Space Complexity (Common)

Problem	Time	Space
Merge Intervals	O(n log n)	O(n)
Activity Selection	O(n log n)	O(1)
Minimum Platforms	O(n log n)	O(1)
Insert Interval	O(n log n)	O(n)
Non-overlapping Intervals	O(n log n)	O(1)

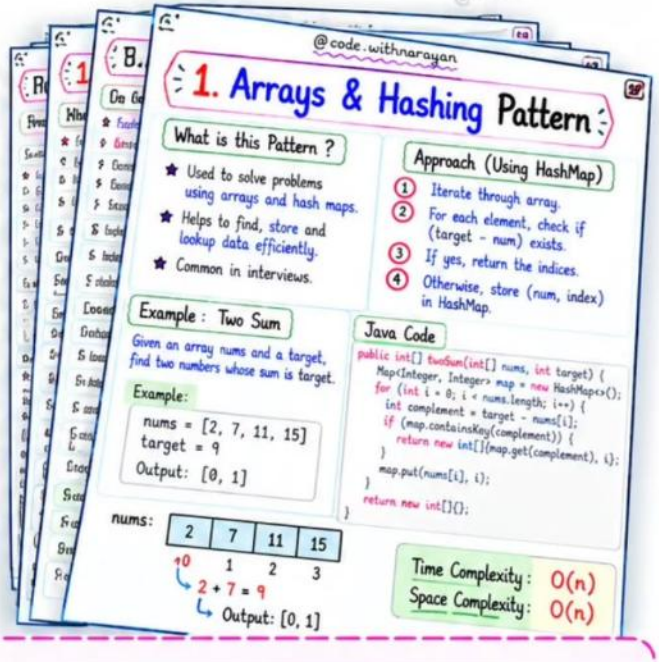
Important Takeaways

- ★ Sort and make greedy choice.
- ★ Check interval overlap carefully.
- ★ Widely used in interview and real-world scheduling problems.

@code.withnarayan

DSA Pattern-Wise Notes Complete

- ✅ 15 Pages (All Important Patterns)
- ✅ Concept + Examples
- ✅ Diagrams & Dry Run
- ✅ Java Code (Clean & Simple)
- ✅ Time & Space Complexity
- ✅ Important Interview Questions
- ✅ Perfect for Placements & Interviews



Follow + Comment "PDF"

@code.withnarayan

Get Link in Your DM

@code.withnarayan